

OBSERVABILITY ON GOOGLE CLOUD — LOGS, METRICS, TRACES, ALERTS

Lab Manual

Four labs on the two Bookshelf services and their pipeline — Cloud Shell and the console only, nothing to code.

Version 1.0 · September 2026 · Cloud Shell gcloud 583.x · Cloud Logging · Monitoring · Trace · region asia-south1

Contents

How to use this manual.....	3
Lab 1 — Cloud Logging: find, filter, count, route, exclude.....	4
Lab 2 — Cloud Monitoring: one page, an uptime check, a burst.....	8
Lab 3 — Alerting and an SLO as measurement.....	10
Lab 4 — Cloud Trace: follow one slow request, name the revision.....	12
Appendix — Environment, versions and the kit.....	14

How to use this manual

The slides teach the ideas and point to a lab; this manual has every step. Every lab works on the same two services — bookshelf-web (public) calling bookshelf-api (internal) — and on what the previous sessions left: bookshelf-api 1.2 with its `FAULT_RATE` setting, which is how the labs make errors happen without touching code. Two steps are console-only because there is no command for them (creating an SLO; opening a trace); everything else has both.

Conventions

- Cloud Shell blocks (teal) are copy-pasteable. Run them one block at a time, in order, from the kit folder.
- `load N` is a helper defined in the first block: `N` page loads of bookshelf-web in parallel — the traffic generator for every lab.
- Metrics and traces lag one to two minutes behind requests; the sheet has sleeps and the steps say when to wait.
- Expected panels show the real shape; counts, ids and timings differ.
- Lab 4 ends without restoring the api unless the trainer says so — the incident session may start from the broken state.

Getting the kit into Cloud Shell

```
$ unzip -q gcp-observability-labs.zip && cd gcp-observability-labs && ls
README.md alert-5xx.json bookshelf-api bookshelf-web dashboard.json labs.sh
teardown.sh
$ PROJECT=$(gcloud config get-value project); R=asia-south1;
AR=$R-docker.pkg.dev/$PROJECT/bookshelf; ME=$(gcloud config get-value account)
$ API_URL=$(gcloud run services describe bookshelf-api --region $R --
format='value(status.url)')
$ WEB_URL=$(gcloud run services describe bookshelf-web --region $R --
format='value(status.url)')
$ load() { for i in $(seq 1 ${1:-50}); do curl -s -o /dev/null "$WEB_URL/?i=$i" &
done; wait; }
$ load 5; echo ok
```

Starting state

Resource	Expected	Check
bookshelf-api	1.2 (<code>FAULT_RATE</code> support), internal ingress, runs as bookshelf-api-min	<code>gcloud run services describe bookshelf-api --region \$R --format='value(spec.template.spec.containers[0].image)' → ...@sha256 or :1.2</code>
bookshelf-web	public; reaches the api	<code>curl -s \$WEB_URL grep -c '' → 4</code>
Cloud Deploy pipeline	bookshelf-api pipeline exists (optional today)	<code>gcloud deploy delivery-pipelines list --region=\$R</code>

If bookshelf-api is not 1.2 (fresh sandbox, or the CI/CD session was not run): the kit carries bookshelf-api/ 1.2 complete — the same code the pipeline deployed. Build and deploy it once, then continue:

```
(cd bookshelf-api && gcloud builds submit --tag $AR/bookshelf-api:1.2)
gcloud run services update bookshelf-api --region $R --image $AR/bookshelf-api:1.2 --quiet
curl -s -H "Authorization: Bearer $(gcloud auth print-identity-token)" $API_URL/books
| head -c 80 # → [{"id": 1, ...
```

What 1.2 adds: `FAULT_RATE` (0.0–1.0, default 0) makes `GET /books` answer HTTP 500 at that rate and logs the line 'simulated fault' — Lab 1 filters on that line and Lab 3 uses the rate to break the service. Learners never edit the code.

Lab 1 — Cloud Logging: find, filter, count, route, exclude

Estimated time	Difficulty	Environment	Kit
25 min	●●○	Cloud Shell + console, region asia-south1	gcp-observability-labs.zip (labs.sh, dashboard.json, alert-5xx.json, bookshelf-api/ 1.2, bookshelf-web/ 1.1)

OBJECTIVE

Traffic with faults generated; the api's request log read, filtered to failures and to the application's own fault line; one page load followed across both services by trace id; a log-based metric on the fault line; a sink routing error entries to a bucket you own; an exclusion dropping health-check noise; bookshelf-web 1.1 deployed; Log Analytics enabled.

PREREQUISITES

- Starting state confirmed; helper load defined.

Steps

Step 1 — Traffic, with 30 % of /books calls failing for a minute

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --update-env-vars FAULT_RATE=0.3
--quiet
load 60; sleep 20; load 60
gcloud run services update bookshelf-api --region $R --remove-env-vars FAULT_RATE --
quiet
```

FAULT_RATE is the setting from the CI/CD session's canary lab. Each update is a new revision; the last one removes it.

Step 2 — The api's request log — one entry per request

Console path: Logging → Logs Explorer → query: resource.type="cloud_run_revision"
resource.labels.service_name="bookshelf-api"

CLOUD SHELL

```
gcloud logging read 'resource.type="cloud_run_revision" AND
resource.labels.service_name="bookshelf-api" AND httpRequest.requestUrl="/books"' \
--limit 5 --format='table(timestamp,httpRequest.status,httpRequest.latency,trace)'
```

EXPECTED

TIMESTAMP	STATUS	LATENCY	TRACE
2026-09-09T14:02:07.118Z	500	0.012s	projects/gcptraining-507817/traces/4f1c9e2b7d3a4c5e8f9a0b1c2d3e4f50
2026-09-09T14:02:07.101Z	200	0.009s	projects/gcptraining-507817/traces/91ab...
2026-09-09T14:02:07.088Z	200	0.011s	projects/gcptraining-507817/traces/c02e...

Nothing was configured to produce this. In the console, the histogram above the results shows when the failures started.

Step 3 — Only the failures — then the application's own line beside them

CLOUD SHELL

```
gcloud logging read 'resource.type="cloud_run_revision" AND
resource.labels.service_name="bookshelf-api" AND httpRequest.status>=500' --limit 3 --
format='value(timestamp,httpRequest.status,trace)'
gcloud logging read 'resource.type="cloud_run_revision" AND
resource.labels.service_name="bookshelf-api" AND textPayload:"simulated fault"' --
limit 3 --format='value(timestamp,textPayload)'
```

EXPECTED

```
2026-09-09T14:02:07.118Z 500 projects/.../traces/4f1c9e2b...
2026-09-09T14:02:07.118Z simulated fault (FAULT_RATE=0.3)
```

The first is the request log (httpRequest fields); the second is stdout (textPayload). Same request, two entries, same timestamp.

Step 4 — One page load across both services — by trace id

CLOUD SHELL

```
TRACE=$(gcloud logging read 'resource.type="cloud_run_revision" AND
resource.labels.service_name="bookshelf-web" AND httpRequest.requestUrl:"/?i=" ' --
limit 1 --format='value(trace)')
echo $TRACE
gcloud logging read "trace=\"$TRACE\"" --
format='table(timestamp,resource.labels.service_name,httpRequest.status,httpRequest.re
questUrl)'
```

EXPECTED

```
projects/gcpttraining-507817/traces/91ab3c...
TIMESTAMP          SERVICE_NAME  STATUS  REQUEST_URL
2026-09-09T14:02:07.140Z  bookshelf-web  200     https://bookshelf-web-.../?i=17
2026-09-09T14:02:07.101Z  bookshelf-api  200     https://bookshelf-api-.../books
```

Two rows means the trace id crossed the service boundary. If you see only the web row, the deployed web does not forward the header yet — do step 8, run load 20, and repeat this step.

Step 5 — A log-based metric on the fault line

Console path: Logging → Log-based metrics → Create metric → Counter, the filter below

CLOUD SHELL

```
gcloud logging metrics create api_simulated_faults --description="bookshelf-api
simulated fault lines" \
--log-filter='resource.type="cloud_run_revision" AND
resource.labels.service_name="bookshelf-api" AND textPayload:"simulated fault"'
gcloud logging metrics list --format='table(name,filter)'
```

EXPECTED

```
Created [api_simulated_faults].
NAME          FILTER
api_simulated_faults  resource.type="cloud_run_revision" AND ... textPayload:"simulated
fault"
```

It counts from now on. It appears on the Lab 2 dashboard as logging.googleapis.com/user/api_simulated_faults.

Step 6 — A sink — route error entries to a bucket you own

Console path: Logging → Log Router → Create sink → Cloud Storage bucket

CLOUD SHELL

```
gcloud storage buckets create gs://bookshelf-logs-$PROJECT --location=$R --uniform-
bucket-level-access
gcloud logging sinks create bookshelf-errors storage.googleapis.com/bookshelf-logs-
$PROJECT \
--log-filter='resource.type="cloud_run_revision" AND severity>=ERROR'
SINK_SA=$(gcloud logging sinks describe bookshelf-errors --
format='value(writerIdentity)')
gcloud storage buckets add-iam-policy-binding gs://bookshelf-logs-$PROJECT --
member=$SINK_SA --role=roles/storage.objectCreator
```

EXPECTED

```
Created [https://logging.googleapis.com/v2/projects/.../sinks/bookshelf-errors].
```

Please remember to grant ``serviceAccount:service-110919260780@gcp-sa-logging.iam.gserviceaccount.com`` the Storage Object Creator role on the bucket.

The sink has its own writer identity; without the binding it silently writes nothing. Objects appear in the bucket about hourly — do not wait for them.

Step 7 — An exclusion — drop health-check noise from _Default

Console path: Logging → Log Router → _Default → Edit sink → Exclusions → Add

CLOUD SHELL

```
gcloud logging sinks update _Default --add-
exclusion=name=healthz,filter='httpRequest.requestUrl:"/healthz"'
gcloud logging sinks describe _Default --format='yaml(exclusions)'
```

EXPECTED

```
exclusions:
- filter: httpRequest.requestUrl:"/healthz"
  name: healthz
```

Excluded entries are not stored and not billed — for this sink. The error sink still receives a failing health check. Watch Logging → Logs Storage over the next hour to see the ingestion rate fall.

Step 8 — bookshelf-web 1.1 — forwards the trace header

CLOUD SHELL

```
(cd bookshelf-web && gcloud builds submit --tag $AR/bookshelf-web:1.1)
gcloud run services update bookshelf-web --region $R --image $AR/bookshelf-web:1.1 --
quiet
load 20; sleep 30
```

EXPECTED

```
SUCCESS    asia-south1-docker.pkg.dev/.../bookshelf/bookshelf-web:1.1
```

The only change in 1.1: the incoming `traceparent` / `X-Cloud-Trace-Context` header is copied onto the call to the api. Repeat step 4 now if it showed one row.

Step 9 — Log Analytics — SQL over the same bucket

Console path: Logging → Log Analytics → paste the query → Run

CLOUD SHELL

```
gcloud logging buckets update _Default --location=global --enable-analytics
# in the console:
SELECT resource.labels.service_name AS service, httpRequest.status AS status,
COUNT(*) AS n
FROM `PROJECT_ID.global._Default._Default`
WHERE timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR) AND
resource.type = 'cloud_run_revision'
GROUP BY 1, 2 ORDER BY 1, 2
```

EXPECTED

service	status	n
bookshelf-api	200	412
bookshelf-api	500	38
bookshelf-web	200	140

Replace `PROJECT_ID`. Analytics applies to entries from now on; if the table is empty, run `load 30` and retry in a minute.

CHECK YOURSELF

- You filtered the request log three ways: by service, by status, by payload.

- One trace id returned rows from both services.
- A metric, a sink with its binding, and an exclusion exist.
- Log Analytics returned a grouped count.

Symptom	Cause	Fix
step 2 returns nothing	no traffic reached the api (web cannot reach it)	curl \$WEB_URL — if 'could not reach', fix web's VPC egress (CI/CD manual troubleshooting)
step 4 shows one row after step 8	cached old revision; traces need fresh requests	load 20, wait 30 s, repeat
sink create: permission	you lack logging.sinks.create (Owner has it)	—

Lab 2 — Cloud Monitoring: one page, an uptime check, a burst

Estimated time	Difficulty	Environment	Kit
25 min	●●○	Cloud Shell + console, region asia-south1	gcp-observability-labs.zip (labs.sh, dashboard.json, alert-5xx.json, bookshelf-api/ 1.2, bookshelf-web/ 1.1)

OBJECTIVE

request_count read in Metrics Explorer as filter → align → reduce; the kit's one-page dashboard created and opened; an uptime check on the web page from outside Google; two bursts observed on the dashboard — instance count stepping, p99 moving, a deployment marker.

PREREQUISITES

- Lab 1 complete (the log-based metric exists).

Steps

Step 1 — One metric in Metrics Explorer, by hand

Read the chart: one line per response class. The 5xx line is Lab 1's faults. Change the aggregation to 'none' and see one line per revision and class — that is what 'one label combination is one series' means.

Console path: Monitoring → Metrics Explorer → Select a metric → Cloud Run Revision → Request Count → Apply; Filter: service_name = bookshelf-api; Rolling window 1 min, function rate; Aggregation sum, group by response_code_class

No CLI step here; the point is the three dropdowns. The dashboard JSON in the next step uses the same names: ALIGN_RATE, REDUCE_SUM, groupByFields.

Step 2 — The dashboard from the kit

Console path: Monitoring → Dashboards → Bookshelf — one page

CLOUD SHELL

```
cat dashboard.json | python3 -c "import json,sys; d=json.load(sys.stdin);
print([t['widget']['title'] for t in d['mosaicLayout']['tiles']])"
gcloud monitoring dashboards create --config-from-file=dashboard.json
gcloud monitoring dashboards list --format='table(displayName,name.basename())'
```

EXPECTED

```
['Request rate (req/s) by service', '5xx rate (req/s) by service', 'Latency p50 /
p95 / p99 - bookshelf-api (ms)', 'Instances by service (active)', 'Simulated faults
logged by bookshelf-api (log-based metric)']
Created [projects/110919260780/dashboards/6d1e...].
DISPLAY_NAME      NAME
Bookshelf — one page  6d1e...
```

Open it. Set the time range to 1 hour. Lab 1's faults are visible on the 5xx tile and the log-based tile. Edit a tile in the console to see the JSON's align/reduce as dropdowns.

Step 3 — An uptime check from outside Google

Console path: Monitoring → Uptime checks → Create

CLOUD SHELL

```
WEB_HOST=${WEB_URL#https://}
gcloud monitoring uptime create "bookshelf-web page" --resource-type=uptime-url --
resource-labels=host=$WEB_HOST,project_id=$PROJECT \
--protocol=https --path=/healthz --port=443 --period=1 --timeout=10 --matcher-
content=ok --matcher-type=contains-string
```

```
gcloud monitoring uptime list-configs --
format='table(displayName,monitoredResource.labels.host,period)'
```

EXPECTED

Created uptime check [bookshelf-web page].

DISPLAY_NAME	HOST	PERIOD
bookshelf-web page	bookshelf-web-110919260780.asia-south1.run.app	60s

Every minute, from several regions, GET /healthz must return 200 with 'ok' in the body. The api is internal — a check cannot reach it, which is correct.

Step 4 — Two bursts, watched on the page**CLOUD SHELL**

```
load 150; sleep 5; load 150
gcloud run services describe bookshelf-api --region $R --
format='value(status.traffic[0].revisionName)'
```

EXPECTED

bookshelf-api-00021-xyz

Wait two minutes, refresh the dashboard: request rate spikes; active instances step from 1 to 2–3 and back; p99 rises during the burst while p50 barely moves — queueing at the instance limit. The vertical marker on the charts is Lab 1's last revision change.

CHECK YOURSELF

- You can name the aligner and reducer behind the request-rate tile.
- The dashboard fits on one screen and shows four signals plus one application metric.
- The uptime check reports passing from three regions (Monitoring → Uptime checks, after a minute).

Symptom	Cause	Fix
dashboard tiles empty	time window too narrow, or metrics lag	1-hour window; refresh after 2 min
uptime check failing	web not public, or /healthz body not 'ok'	curl -s \$WEB_URL/healthz → ok; check the org-policy override (IAM session)
p99 tile flat during burst	burst too small for the instance limit	load 300 twice

Lab 3 — Alerting and an SLO as measurement

Estimated time	Difficulty	Environment	Kit
25 min	...	Cloud Shell + console, region asia-south1	gcp-observability-labs.zip (labs.sh, dashboard.json, alert-5xx.json, bookshelf-api/ 1.2, bookshelf-web/ 1.1)

OBJECTIVE

An email notification channel; the kit's threshold policy created from JSON and read in the console; a 99 % availability SLO on bookshelf-api with a fast-burn alert; the api broken for three minutes; the incident opened and the email received; the api restored; the incident closed; the error budget read.

PREREQUISITES

- Lab 2 complete.

Steps

Step 1 — A notification channel — your email

Console path: Monitoring → Alerting → Edit notification channels → Email → Add new

CLOUD SHELL

```
CHANNEL=$(gcloud beta monitoring channels create --display-name="grad email" --
type=email --channel-labels=email_address=$ME --format='value(name)')
echo $CHANNEL
```

EXPECTED

```
projects/gcptesting-507817/notificationChannels/1234567890123456789
```

Step 2 — The threshold policy from the kit

Console path: Monitoring → Alerting → Policies → the new policy → read Conditions and Documentation

CLOUD SHELL

```
sed "s#CHANNEL_NAME#CHANNEL#" alert-5xx.json > /tmp/alert.json
grep -E 'thresholdValue|duration|perSeriesAligner' /tmp/alert.json
gcloud monitoring policies create --policy-from-file=/tmp/alert.json
gcloud monitoring policies list --
format='table(displayName,enabled,conditions[0].displayName)'
```

EXPECTED

```
"thresholdValue": 0.05,
"duration": "120s",
"aggregations": [{"alignmentPeriod": "60s", "perSeriesAligner": "ALIGN_RATE",
...
Created alert policy [projects/.../alertPolicies/...].
DISPLAY_NAME      ENABLED  DISPLAY_NAME
bookshelf-api - 5xx responses for 2 minutes  True    5xx rate > 0.05 req/s (3 per
minute) for 2 min
```

Read the documentation block in the console: it is what the responder sees at 2 a.m. — 'first check: a release or IAM change in the last 30 minutes'.

Step 3 — An SLO — 99 % availability, rolling 28 days — and a fast-burn alert

There is no gcloud command for SLOs; the console builds the same API object. Create it now so it has data before step 4.

Console path: Cloud Run → bookshelf-api → SLOs tab → Create SLO → Availability · Request-based → Goal 99 % · Rolling 28 days → Save. Then on the SLO: Create alerting policy → Burn rate · Lookback 60 min · Threshold 10 → notification: grad email → Save.

The SLI is computed from `run.googleapis.com/request_count`: good = non-5xx. The page shows the current SLI, the budget remaining and a burn-down chart once a few minutes of data exist.

Step 4 — Break it — every /books call fails for three minutes

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --update-env-vars FAULT_RATE=1.0
--quiet
load 60; sleep 60; load 60; sleep 60; load 60
```

Watch Monitoring → Alerting → Incidents. The threshold policy needs two consecutive minutes over 0.05 req/s of 5xx, then notifies within about a minute; the email subject is '[ALERT] bookshelf-api — 5xx responses for 2 minutes'. The burn-rate alert follows once the SLO's hourly error rate crosses 10 % — with a 100 % error rate that is minutes.

Step 5 — Restore, close, and read the budget

Console path: Cloud Run → bookshelf-api → SLOs → the availability SLO

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --remove-env-vars FAULT_RATE --
quiet
load 60
```

EXPECTED

```
(console) Error budget remaining: 87 % — a step down at 14:31, the three minutes of
failures
(console) Incidents: 'bookshelf-api — 5xx responses for 2 minutes' — Closed (or
closing: auto-close 30 min after the condition clears)
```

Three minutes of total failure at this traffic spent about a tenth of a 28-day budget at 99 %. Say the number out loud: that is what the promise costs.

CHECK YOURSELF

- An email arrived (check spam), or at least the incident opened.
- The SLO page shows a dent.
- You can point at the JSON and say which line is the duration and why it is not zero.

Symptom	Cause	Fix
no incident after 4 minutes	5xx rate below 0.05 req/s — too little traffic	load 120 twice more; check the 5xx tile on the dashboard
incident but no email	channel address wrong, or mail delayed	gcloud beta monitoring channels list; wait; spam folder
SLO chart empty	created moments ago	wait 5 min; the burn-rate alert needs data too

Lab 4 — Cloud Trace: follow one slow request, name the revision

Estimated time	Difficulty	Environment	Kit
20 min	●●○	Cloud Shell + console, region asia-south1	gcp-observability-labs.zip (labs.sh, dashboard.json, alert-5xx.json, bookshelf-api/ 1.2, bookshelf-web/ 1.1)

OBJECTIVE

The api made slow by configuration — one instance, one request at a time — and a burst; the slowest traces of the web page found in Trace Explorer; a waterfall read; the jump from a span to the log entries of that trace; and the culprit revision named with its creation time and what changed. The lab stops there.

PREREQUISITES

- Lab 1 step 8 done (web 1.1 forwards the trace header).

Steps

Step 1 — Make the api slow without touching code

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --concurrency 1 --max-instances 1
--quiet
load 40; sleep 90
```

Forty page loads, one instance, one request at a time: the later requests queue. This is real latency, not a sleep — the kind a bad sizing change produces.

Step 2 — The slowest traces

The waterfall shows two spans: bookshelf-web GET / on top, bookshelf-api GET /books beneath it, with most of the time in the api span. Each span shows the revision that served it.

Console path: Trace → Trace Explorer → filter: Service = bookshelf-web (or span name GET /) → sort by latency → open the slowest

EXPECTED

```
bookshelf-web    GET /      [Progress Bar] 3 820 ms  bookshelf-
web-00006-...
bookshelf-api    GET /books [Progress Bar] 3 790 ms  bookshelf-
api-00023-...   ← here
```

Cloud Run samples traces; a burst of 40 leaves plenty. If the list is empty, wait another minute and refresh.

Step 3 — From the span to the logs

Console path: in the trace: 'Logs for this trace' (or copy the trace id)

CLOUD SHELL

```
# paste the trace id from the console:
gcloud logging read 'trace="projects/'$PROJECT'/traces/TRACE_ID"' --
format='table(timestamp,resource.labels.service_name,resource.labels.revision_name,htt
pRequest.status,httpRequest.latency)'
```

EXPECTED

TIMESTAMP	SERVICE_NAME	REVISION_NAME	STATUS	LATENCY
2026-09-09T14:52:31.902Z	bookshelf-web	bookshelf-web-00006-abc	200	3.820s
2026-09-09T14:52:31.880Z	bookshelf-api	bookshelf-api-00023-xyz	200	3.790s

Same trace id, both services, both revisions named. The api took the time; the web only waited.

Step 4 — Name the culprit — and what changed

CLOUD SHELL

```
gcloud run revisions list --service bookshelf-api --region $R --
format='table(name,active,metadata.creationTimestamp,spec.containerConcurrency)' |
head -4
gcloud logging read 'logName:"cloudaudit.googleapis.com%2Factivity" AND
protoPayload.serviceName="run.googleapis.com" AND
protoPayload.resourceName:"bookshelf-api"' --freshness=30m --limit 3 --
format='table(timestamp,protoPayload.authenticationInfo.principalEmail,protoPayload.me
thodName)'
```

EXPECTED

NAME	ACTIVE	CREATION_TIMESTAMP	CONTAINER_CONCURRENCY
bookshelf-api-00023-xyz	yes	2026-09-09T14:49:58Z	1
bookshelf-api-00022-uvw		2026-09-09T14:35:12Z	80
TIMESTAMP	PRINCIPAL_EMAIL	METHOD_NAME	
2026-09-09T14:49:58Z	rohan@example.com	google.cloud.run.v1.Services.ReplaceService	

The fact: revision 00023, created 14:49:58 by you, concurrency 1 — the slowness started with it. That is where this session ends. What to do with the fact is the incident session.

Step 5 — Restore the sizing (only if the incident session is NOT next)

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --concurrency 80 --max-instances
3 --quiet
```

Ask the trainer. The incident session can start from the broken state.

CHECK YOURSELF — AND LEAVE THE ALERT, DASHBOARD AND SLO IN PLACE

- You found the slowest trace and read two spans.
- The log query by trace id returned both services with revision names.
- You can say: revision, creation time, who, what changed.

Appendix — Environment, versions and the kit

Versions and facts verified 2026-09-09

Item	Value
gcloud	583.x (Cloud Shell); gcloud monitoring uptime/dashboards/policies GA; gcloud beta monitoring channels
Cloud Run request log	logName run.googleapis.com/requests; httpRequest.*; trace = projects/P/traces/ID; resource cloud_run_revision
Cloud Run metrics	run.googleapis.com/request_count (response_code_class), request_latencies (distribution), container/instance_count (state), container/cpu memory/utilizations, container/startup_latencies, container/billable_instance_time
Alert policy	conditionThreshold, ALIGN_RATE 60 s, REDUCE_SUM, > 0.05 req/s of 5xx for 120 s; email channel
SLO	console-created on the Cloud Run service; availability request-based; burn-rate alert lookback 60 min, threshold 10
Trace propagation	traceparent / X-Cloud-Trace-Context forwarded by bookshelf-web 1.1; Cloud Run creates request spans, sampled
bookshelf-api	1.2 from the CI/CD kit — FAULT_RATE env var

The kit

File	Purpose
labs.sh	Labs 1–4 as one command sheet
bookshelf-api/	version 1.2 (FAULT_RATE), supplied complete — only needed if the CI/CD session did not leave it deployed
dashboard.json	the one-page dashboard
alert-5xx.json	the threshold policy; CHANNEL_NAME substituted in Lab 3
bookshelf-web/	1.1, supplied complete — trace-header forwarding
teardown.sh	removes this session's additions; keeps the services

What this session leaves behind — the incident session detects on it

- alert policy 'bookshelf-api — 5xx responses for 2 minutes' → grad email
- dashboard 'Bookshelf — one page'; uptime check 'bookshelf-web page'
- SLO 99 % availability on bookshelf-api with a fast-burn alert
- log-based metric api_simulated_faults; sink bookshelf-errors; exclusion healthz; Log Analytics on _Default
- bookshelf-web 1.1; bookshelf-api possibly left at concurrency 1 (Lab 4) — the trainer decides